IU INTERNATIONALE HOCHSCHULE

Course: DLBAIPNLP01 - Project: NLP

Task 2 - Text Classification for Spam Detection

TEXT CLASSIFICATION FOR EMAIL SPAM DETECTION USING NATURAL LANGUAGE PROCESSING

Project Report

Student: Olusola Owoyemi

Submission date: October 2026

GitHub repository link is provided in Appendix A.

Abstract

Email remains an important form of digital communication for individuals and organisations, but its widespread use also enables the distribution of unsolicited and potentially harmful messages. This project develops and evaluates a supervised Natural Language Processing (NLP) system for binary email spam classification using the Enron Spam dataset. The original dataset contained 33,716 labelled emails. Duplicate message content was removed as a data-quality measure, leaving 30,494 unique emails for the final experiments. The text was cleaned and preprocessed through stop-word removal and lemmatization, then represented numerically with Term Frequency-Inverse Document Frequency (TF-IDF). A Multinomial Naive Bayes classifier was trained using an 80:20 stratified training and test split. On the held-out test set of 6,099 emails, the model achieved 98.79% accuracy and correctly classified 6,025 messages. The complete pipeline was also evaluated on progressively larger datasets of 1,000, 5,000, 10,000, 20,000 and 30,494 emails. Accuracy increased overall from 95.00% on the smallest dataset to 98.79% on the full dataset, although a small decrease occurred at 20,000 emails. On this corpus, the results show that a classical NLP pipeline based on TF-IDF and Multinomial Naive Bayes can classify spam effectively, while manual testing also highlights limitations for very short messages and for generalization beyond the Enron domain.

Keywords: Natural Language Processing; email spam detection; text classification; TF-IDF; Multinomial Naive Bayes; Enron Spam dataset

1. Introduction

1.1 Background and Motivation

Electronic mail is a central communication medium for personal, professional and commercial activities. Its low distribution cost and global reach also make it attractive for the mass delivery of unsolicited messages. Spam may consist of unwanted advertising, deceptive offers, phishing attempts or other content that creates inconvenience and potential security risks for recipients. As the volume of digital communication grows, manual inspection of every message is impractical, which motivates the development of automated filtering systems.

Natural Language Processing (NLP) provides computational methods for analysing and representing human language. One major NLP application is text classification, in which a document is assigned to one or more predefined categories. Spam detection can therefore be framed as a binary text-classification problem: given the text of an email, determine whether it is legitimate or spam. This framing is consistent with established NLP practice, where raw text is transformed into numerical features and supervised learning is used to infer class membership from labelled examples (Jurafsky & Martin, 2013).

The project focuses on a transparent classical machine-learning pipeline rather than a more computationally expensive deep-learning architecture. The selected approach makes each stage of the system - cleaning, preprocessing, feature representation, classification and evaluation - directly observable and suitable for systematic experimentation.

1.2 Problem Statement

Raw email messages cannot be passed directly to most classical machine-learning algorithms. They contain variable-length text, formatting artefacts, punctuation and high-frequency words that may not be useful for discrimination. The problem therefore requires both linguistic preprocessing and a numerical representation of text. A second challenge is evaluation: a model that performs well on messages it has already seen may still fail on new emails. The system must therefore be trained on one portion of the data and evaluated on a separate held-out test set.

A further concern is how the quantity of available training data affects classification performance. For that reason, the same NLP pipeline is evaluated on progressively larger datasets instead of relying on a single full-dataset result.

1.3 Aim and Objectives

The aim of this project is to develop and evaluate an NLP system that classifies email messages as either legitimate or spam.

- Obtain and inspect a labelled, text-based email spam dataset.
- Clean and preprocess the email text, including stop-word removal and lemmatization.
- Encode the processed text into numerical features suitable for machine learning.
- Train a supervised classifier using labelled legitimate and spam emails.
- Evaluate the model on a separate test dataset using appropriate classification metrics.
- Demonstrate classification of previously unseen email text as legitimate or spam.
- Evaluate the complete system on progressively larger datasets and analyse the resulting performance trend.

1.4 Research Questions

The implementation and experiments are guided by two practical research questions:

- RQ1: How effectively can TF-IDF features combined with Multinomial Naive Bayes distinguish legitimate emails from spam in the Enron Spam dataset?
- RQ2: How does the size of the available dataset affect the accuracy and related performance metrics of the classifier?

1.5 Structure of the Report

Section 2 introduces the relevant theoretical background and related work. Section 3 describes the dataset, preprocessing pipeline, feature extraction, model training and experimental design. Section 4 presents the quantitative results. Section 5 discusses the findings, errors and limitations, and Section 6 concludes the report and outlines possible future work.

2. Theoretical Background and Related Work

2.1 Natural Language Processing and Text Classification

NLP is concerned with computational processing of human language and includes tasks such as machine translation, information retrieval, sentiment analysis and text classification (Jurafsky & Martin, 2013). In supervised text classification, a model is trained using documents for which the desired class labels are already known. The learned model is then used to predict the class of unseen documents.

Spam detection is a natural example of supervised binary classification because historical email corpora can be labelled as legitimate or spam. Recent literature demonstrates that spam and general text classification can be approached with both classical methods and neural architectures. Elakkiya, Selvakumar and Leela Velusamy (2021), for example, present a deep-learning framework for textual social-spam detection, while Huan et al. (2022) investigate text classification using a convolutional and bidirectional long short-term memory architecture. These approaches demonstrate the breadth of methods available, but they also motivate the value of a simpler interpretable baseline against which more complex systems can later be compared.

2.2 Text Preprocessing

Text collected from real communication systems contains variation that can increase the dimensionality of a learning problem. Common preprocessing operations include case normalization, tokenization, removal of selected high-frequency words and morphological normalization (Jurafsky & Martin, 2013). In this project, preprocessing is used to produce a more consistent representation of each email while retaining the textual information needed for classification.

The preprocessing stage explicitly includes stop-word removal. Stop words are frequent function words such as "the", "and", "is" and "you". Removing them can reduce weakly informative vocabulary, although the later manual tests show that aggressive removal can also reduce context in very short messages. Lemmatization was selected as the morphological-normalization technique because the task permits either stemming or lemmatization, and the lemmatized output is easier to interpret in the resulting feature space (Jurafsky & Martin, 2013).

2.3 TF-IDF Feature Representation

Most classical machine-learning algorithms require fixed-length numerical inputs. Term Frequency-Inverse Document Frequency (TF-IDF) is a widely used document representation that weights terms according to their frequency within a document and their prevalence across the wider collection (Manning, Raghavan, & Schütze, 2008). A simplified representation is:

$$\textbf{TF-IDF}(t, d) = \textbf{TF}(t, d) \times \textbf{IDF}(t)$$

Terms that occur frequently in a particular email can receive higher weight, while terms that appear across much of the corpus are down-weighted. The resulting sparse matrix represents emails as rows and vocabulary features as columns (Manning et al., 2008). In this implementation, the vectorizer is fitted only on the training data and then applied unchanged to the test data so that information from the held-out corpus does not influence feature generation.

2.4 Multinomial Naive Bayes

Naive Bayes is a probabilistic family of supervised classifiers based on Bayes' theorem. Multinomial Naive Bayes is widely used for document classification with high-dimensional, non-negative text features (Manning et al., 2008). The model estimates how strongly observed features are associated with each class and combines those associations with class probabilities to predict the most likely label.

The "naive" assumption treats features as conditionally independent given the class. Natural-language terms are not truly independent, but the simplification can still work well for bag-of-words-style document representations (Manning et al., 2008). Its low computational cost also made it practical to retrain the classifier independently for each dataset-size experiment in this project.

2.5 Evaluation Metrics

Accuracy alone can hide important classification errors, so the evaluation also reports precision, recall and F1-score (Jurafsky & Martin, 2013). For the spam class, precision measures how often a spam prediction is correct, whereas recall measures how many of the actual spam messages are detected. The F1-score combines precision and recall using their harmonic mean.

$$\text{Precision} = TP / (TP + FP) \qquad \text{Recall} = TP / (TP + FN) \qquad F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

A confusion matrix is also used to display correct and incorrect predictions for both classes, making false-positive and false-negative behaviour explicit.

3. Dataset and Methodology

3.1 Dataset Selection and Source

For this project, the Enron Spam dataset distributed on Kaggle by Bert van Keulen was selected because it contains raw email text rather than only pre-engineered numerical attributes. This made it possible to perform the required NLP preprocessing directly on the message content. The source file used in the implementation is Enron_Txt_fn.csv (van Keulen, n.d.).

The original dataset contains 33,716 records with three columns: label, email and filename. The target label is binary, where 0 represents legitimate email and 1 represents spam. The original class distribution contains 16,545 legitimate emails and 17,171 spam emails.

Table 1. Dataset summary before and after duplicate handling

Measure	Value	Comment
Original records	33,716	All labelled email records
Original legitimate (label 0)	16,545	49.07% of original data
Original spam (label 1)	17,171	50.93% of original data
Duplicate email texts	3,222	Repeated message content
Conflicting duplicate labels	0	No identical text had both labels
Final unique records	30,494	Working dataset after duplicate removal
Final legitimate	15,910	52.17% of working data
Final spam	14,584	47.83% of working data

3.2 Initial Inspection and Data Quality

The dataset was inspected using Pandas before any modelling was performed. The inspection covered dataset dimensions, column names, missing values, label frequencies, representative legitimate and spam messages, and duplicated email content. No missing values were present in the three columns required for the project.

A total of 3,222 duplicate email texts were found. Identical messages did not contain conflicting labels. Duplicate content was removed before the train/test split so that an identical email could not appear in both training and testing sets. This reduced the working dataset to 30,494 unique messages. Duplicate removal was an additional data-quality decision rather than a separate requirement of the classifier.

3.3 Basic Text Cleaning

The email field contained raw strings with dataset-specific list-like wrappers, line breaks, punctuation, numbers and inconsistent capitalization. A regular-expression based cleaning function was therefore applied. The procedure converted all text to lowercase, removed outer wrapper characters, replaced line breaks with spaces, removed punctuation, digits and other special characters, and collapsed repeated whitespace.

The original email column was retained and a separate clean_email column was created. Keeping both versions made the transformation auditable and allowed samples to be inspected before proceeding to further NLP preprocessing.

3.4 Stop-word Removal and Lemmatization

Cleaned messages were tokenized using whitespace splitting. English stop words from NLTK were removed, and the remaining tokens were passed through the WordNet lemmatizer. The tokens were then joined back into a processed_email string. Lemmatization was chosen instead of stemming because the project required one of the two techniques and lemmatized output is generally easier to interpret.

A potential consequence of this preprocessing is the loss of contextual words in very short messages. This issue was observed during later manual testing and is discussed in Section 5.

3.5 Processing Pipeline

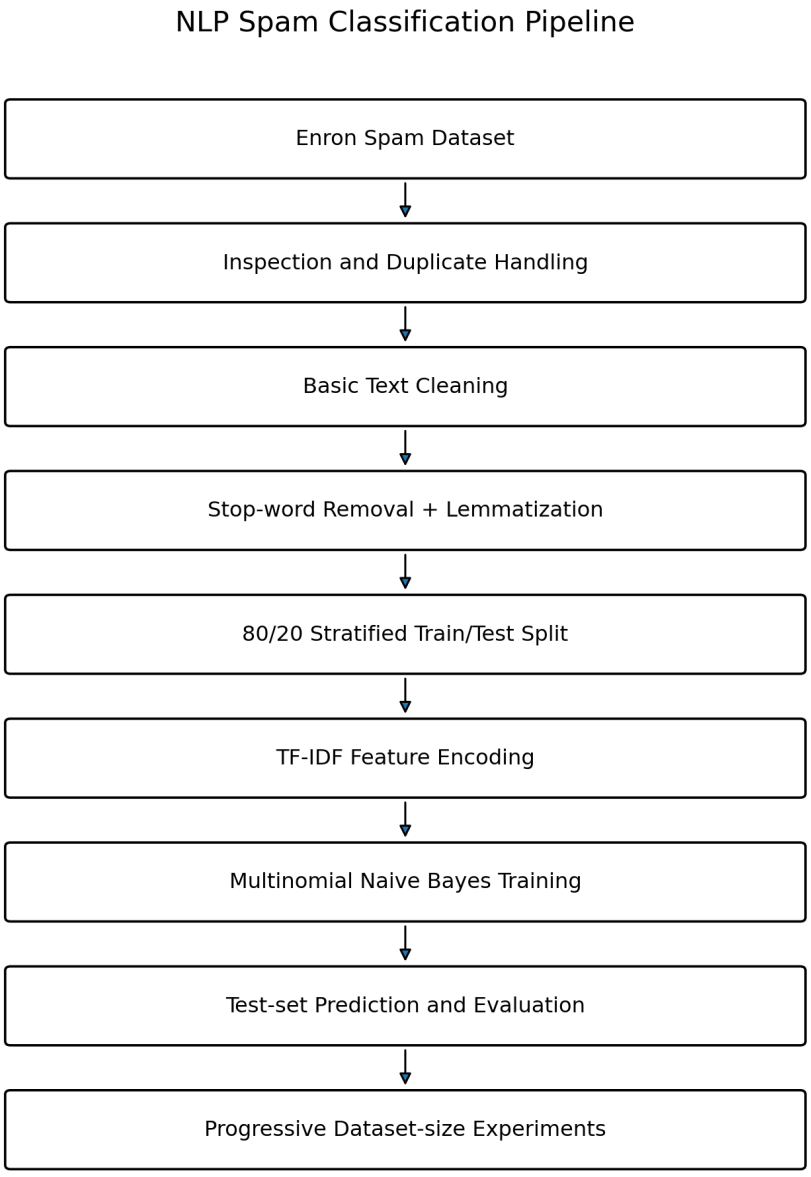


Figure 1. NLP spam-classification pipeline used in the project

3.6 Train/Test Split

The processed messages were divided into input features X and target labels y: X contained the processed email text and y contained the corresponding legitimate/spam labels. Scikit-learn's train_test_split function was used with an 80:20 split. The value random_state=42 made the split reproducible, while stratify=y preserved approximately the same class proportions in both subsets.

For the final working dataset this procedure produced 24,395 training emails and 6,099 test emails. The test set was not used to fit either the TF-IDF vectorizer or the classifier.

3.7 TF-IDF Encoding

A Scikit-learn TfidfVectorizer was created using its default configuration. Applying fit_transform to X_train learned the training vocabulary and inverse-document-frequency statistics while converting the training emails into a sparse numerical matrix. The fitted vectorizer was then applied to X_test with transform only. Keeping the fitting stage confined to the training set avoids data leakage into the held-out evaluation data.

With the full working dataset, the training matrix contained 119,215 TF-IDF features. Each training email therefore became a sparse numerical vector in the same feature space.

3.8 Model Training and Prediction

A Multinomial Naive Bayes classifier was instantiated using the default Scikit-learn configuration and trained on the TF-IDF representation of the training emails together with y_train. The trained model then produced y_pred for the held-out test matrix. Evaluation compared these predictions with y_test only after prediction had been completed.

A reusable predict_email function was also implemented for new raw email text. The function applies the same cleaning and NLP preprocessing used during model development, transforms the new message with the already-fitted TF-IDF vectorizer, and passes the resulting vector to the trained classifier. The function returns either “Spam” or “Legitimate”.

3.9 Progressive Dataset-size Experiments

To evaluate the effect of corpus size, the entire pipeline was repeated on datasets of 1,000, 5,000, 10,000, 20,000 and 30,494 emails. Each subset was selected with stratification to preserve the class distribution. Each experiment then created its own 80:20 train/test split, a new TF-IDF vectorizer and a new Multinomial Naive Bayes classifier. Reinitializing the vectorizer and model prevented later experiments from inheriting information learned in earlier runs.

For every experiment, dataset size, training size, test size, number of TF-IDF features, accuracy, weighted precision, weighted recall and weighted F1-score were recorded.

3.10 Implementation Environment

The implementation was developed in Google Colab using Python. Pandas was used for data loading and inspection; Python regular expressions were used for basic text cleaning; NLTK supplied English stop words and WordNet lemmatization; Scikit-learn provided train/test splitting, TF-IDF encoding, Multinomial Naive Bayes and evaluation metrics; Matplotlib was used for visualization; and Joblib was used to save the trained model and vectorizer.

Table 2. Main implementation choices

Component	Implementation
Development environment	Google Colab / Python
Text cleaning	Python re + string operations
Stop-word list	NLTK English stop words
Morphological normalization	WordNet lemmatization
Train/test split	80/20 stratified; random_state=42
Text representation	TfidfVectorizer
Classifier	MultinomialNB
Primary metric	Accuracy
Additional metrics	Precision, recall, F1-score, confusion matrix

4. Results

4.1 Full-dataset Performance

The final model was trained on 24,395 emails and evaluated on a held-out set of 6,099 emails. It achieved 98.79% accuracy, corresponding to 6,025 correct predictions and 74 incorrect predictions. Table 3 reports the class-wise precision, recall and F1-scores; overall accuracy is reported separately because it is a single dataset-level measure.

Table 3. Class-wise classification metrics for the full held-out test set

Class	Precision	Recall	F1-score	Support
Legitimate (0)	0.98	0.99	0.99	3,182
Spam (1)	0.99	0.98	0.99	2,917
Macro average	0.99	0.99	0.99	6,099
Weighted average	0.99	0.99	0.99	6,099

For the spam class, precision of approximately 0.99 indicates that very few legitimate emails were incorrectly flagged as spam. Recall of approximately 0.98 indicates that a small number of actual spam emails were missed. Performance for the legitimate class was similarly high, with recall of approximately 0.99.

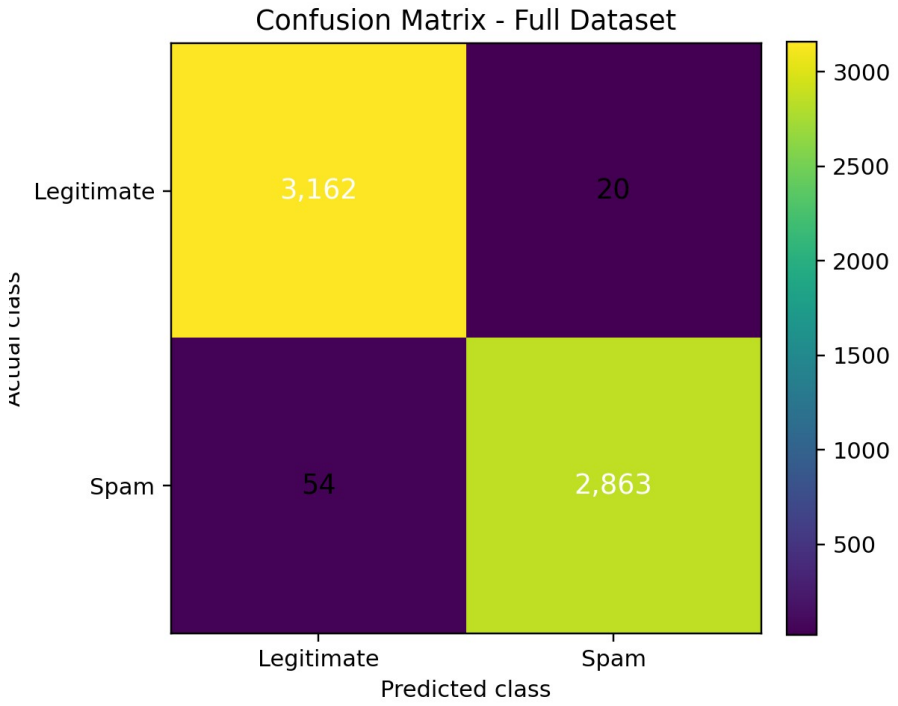


Figure 2. Confusion matrix for the full held-out test set

The confusion matrix shows 3,162 legitimate emails correctly classified as legitimate and 2,863 spam emails correctly classified as spam. Twenty legitimate emails were falsely flagged as spam, while 54 spam emails were classified as legitimate. The model therefore produced fewer false positives than false negatives in this test set.

4.2 Progressive Dataset-size Results

Table 4. Performance across progressively larger datasets

Dataset	Train	Test	TF-IDF features	Accuracy	Weighted precision	Weighted recall	Weighted F1
1,000	800	200	17,687	95.00%	95.28%	95.00%	94.98%
5,000	4,000	1,000	46,566	97.70%	97.70%	97.70%	97.70%
10,000	8,000	2,000	65,942	98.40%	98.40%	98.40%	98.40%
20,000	16,000	4,000	95,355	98.15%	98.16%	98.15%	98.15%
30,494	24,395	6,099	119,215	98.79%	98.79%	98.79%	98.79%

The 1,000-email experiment achieved 95.00% accuracy. Expanding the dataset to 5,000 emails increased accuracy to 97.70%, and 10,000 emails produced 98.40%. The 20,000-email experiment showed a small decrease to 98.15%, after which the full 30,494-email dataset achieved the highest observed accuracy of 98.79%. The overall pattern therefore supports a positive relationship between available data and model performance, although the relationship is not strictly monotonic. Precision, recall and F1 in Table 4 are weighted averages across the two classes.

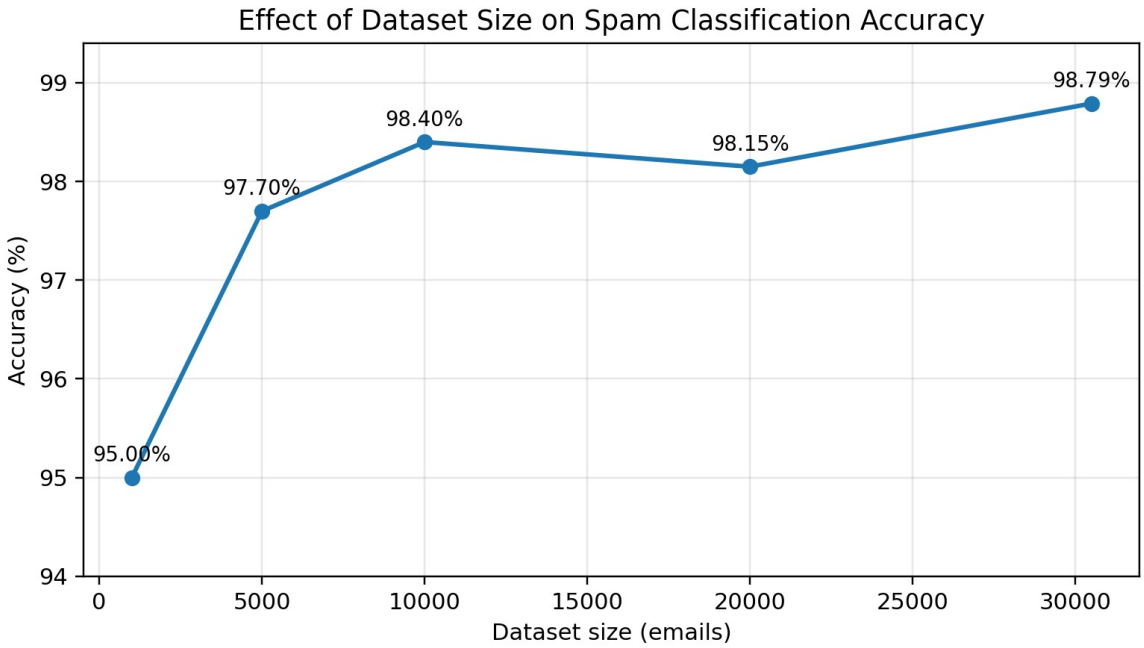


Figure 3. Effect of dataset size on classification accuracy

The number of TF-IDF features also increased substantially with dataset size, from 17,687 features in the 1,000-email experiment to 119,215 features in the full-data experiment. This reflects the greater vocabulary diversity encountered as more unique emails are introduced.

4.3 Classification of New Messages

The predict_email function successfully provided binary predictions for new raw messages by reusing the preprocessing pipeline, fitted TF-IDF vectorizer and trained classifier. Obvious promotional or prize-oriented messages were generally classified as spam, while ordinary project and meeting correspondence was classified as legitimate in manual checks.

A useful diagnostic case occurred with the very short message “I love you”, which the classifier labelled as spam. After stop-word removal, the message is reduced largely to the term “love”, leaving little contextual information. The result illustrates that high aggregate accuracy does not imply correct classification of every possible message and provides a concrete example for analysing model limitations.

5. Discussion

5.1 Interpretation of Overall Performance

On the held-out Enron test set, the selected classical NLP pipeline achieved 98.79% accuracy. Precision, recall and F1-scores were also high for both classes, so the result is not explained by a model that simply favours one label. The near-balanced working dataset and stratified split meant that both legitimate and spam messages were well represented in the evaluation.

The model’s spam precision is slightly stronger than its spam recall. In operational terms, this means that when the model predicts spam it is very often correct, but a small number of spam messages still pass through as legitimate. Depending on the intended use case, a system designer might later choose to adjust

decision thresholds or compare alternative models if missing spam is considered more costly than incorrectly flagging legitimate correspondence.

5.2 Effect of Dataset Size

The progressive experiments show how the same pipeline behaves under different data-volume scenarios. Accuracy rose from 95.00% with 1,000 emails to 98.40% with 10,000 emails, suggesting that additional examples exposed the classifier to a broader range of legitimate and spam vocabulary. The best result, 98.79%, was obtained with the full 30,494-email dataset.

The decrease from 98.40% at 10,000 messages to 98.15% at 20,000 messages is not contradictory. Each experiment uses a different stratified subset and test set, so difficulty can vary slightly across experiments. Model performance therefore should not be expected to rise perfectly at every step. The appropriate interpretation is the overall trend rather than a strictly monotonic sequence.

5.3 Error Analysis

The confusion matrix contains two important error types. The 20 false positives are legitimate emails incorrectly classified as spam. In a real filtering system, false positives can be costly because users may fail to see important correspondence. The 54 false negatives are spam emails classified as legitimate, representing unwanted messages that escape filtering. The model makes relatively few errors of either type, but the difference shows that spam recall remains a useful area for improvement.

The manual “I love you” example provides a second form of error analysis. Because preprocessing removes common stop words, very short messages may lose much of their original grammatical context. The resulting TF-IDF vector can be dominated by one or two tokens whose learned statistical association with spam may not match the human meaning of the full message.

5.4 Limitations and Threats to Generalization

The strongest limitation is domain dependence. The reported 98.79% accuracy is a result on held-out examples from the Enron Spam dataset. It should not be interpreted as evidence that the same accuracy will necessarily be achieved on current consumer email, financial-services communication, social-media messages or other corpora. Vocabulary, sender behaviour and spam strategies vary across domains and over time.

The preprocessing design also removes digits, punctuation and special characters. These elements can carry useful spam signals, such as currency amounts, telephone numbers, repeated exclamation marks or unusual URL patterns. Removing them simplifies the text representation but may discard information that a more advanced model could exploit.

TF-IDF and Multinomial Naive Bayes model statistical term associations rather than deep contextual semantics. Word order and longer-range relationships are largely ignored. As a result, the classifier can struggle when meaning depends on context, negation, unusual phrasing or a very small number of tokens.

Finally, the experiments use a single fixed random state and one 80:20 split for each selected corpus size. Repeated cross-validation would provide a more robust estimate of variability, but it was not required for the present implementation.

5.5 Reproducibility and Practical Value

Several choices were made to keep the experiments reproducible: sampling and splitting use a fixed random state, the same preprocessing functions are applied consistently, TF-IDF is fitted only on training data, and

every progressive experiment creates a fresh vectorizer and classifier. The associated GitHub repository contains the implementation, result files and generated figures.

The project also provides a clear baseline for future development. Because each stage is modular, later work can compare alternative preprocessing decisions, additional feature representations or different supervised algorithms while retaining the same evaluation structure.

6. Conclusion and Outlook

This project developed a Natural Language Processing system for binary email spam classification using the Enron Spam dataset. The implementation covered the complete workflow from dataset inspection to preprocessing, TF-IDF feature extraction, supervised model training, held-out testing, new-message prediction and progressive dataset-size evaluation.

After removal of duplicate message content, 30,494 unique emails remained. Stop words were removed and WordNet lemmatization was applied before TF-IDF encoding. A Multinomial Naive Bayes classifier trained on 24,395 emails achieved 98.79% accuracy on a separate test set of 6,099 emails, correctly classifying 6,025 messages. The confusion matrix contained 20 false positives and 54 false negatives, while both classes achieved approximately 0.99 F1-score.

The progressive experiments showed that performance generally improved as the amount of available data increased. Accuracy rose from 95.00% with 1,000 emails to 98.79% with the full dataset. This supports the conclusion that the classifier benefits from greater exposure to diverse textual patterns, although the small variation between 10,000 and 20,000 messages also shows that performance is not guaranteed to increase at every individual step.

Future work could compare the current model with Support Vector Machines, logistic regression or neural text-classification approaches. Cross-validation and evaluation on additional email corpora would provide stronger evidence of generalization. Alternative preprocessing could also retain information currently removed, such as numbers, punctuation, URL characteristics or n-gram context. These extensions would build on the current system while preserving the transparent experimental baseline established in this report.

References

Elakkiya, E., Selvakumar, S., & Leela Velusamy, R. (2021). TextSpamDetector: Textual content based deep learning framework for social spam detection using conjoint attention mechanism. Journal of Ambient Intelligence and Humanized Computing, 12(10), 9287-9302.

Huan, H., Guo, Z., Cai, T., & He, Z. (2022). A text classification method based on a convolutional and bidirectional long short-term memory model. Connection Science, 34(1), 2108-2124.

Jurafsky, D., & Martin, J. H. (2013). Speech and language processing: An introduction to natural language processing, computational linguistics, and speech recognition (2nd ed.). Pearson Prentice Hall.

Manning, C. D., Raghavan, P., & Schütze, H. (2008). Introduction to information retrieval. Cambridge University Press.

van Keulen, B. (n.d.). Enron Spam [Data set]. Kaggle. https://www.kaggle.com/datasets/bertvankeulen/enron-spam

Appendix A. Repository and Project Artifacts

The complete implementation is maintained in a GitHub repository. The repository contains the Google Colab/Jupyter notebook, requirements file, experimental result CSV, generated figures, saved model/vectorizer files (where included), and this project report.

GitHub repository: https://github.com/oluseyisholly/enron-spam-classification

Dataset source: https://www.kaggle.com/datasets/bertvankeulen/enron-spam

Main reproducibility settings: random_state=42; 80% training / 20% testing; stratified splitting; TF-IDF fitted on training data only; Multinomial Naive Bayes classifier.

Repository structure

- `README.md`
- `requirements.txt`
- `notebooks/enron_spam_classification.ipynb`
- `results/dataset_size_experiment_results.csv`
- `results/dataset_size_vs_accuracy.png`
- `results/confusion_matrix.png`
- `models/spam_classifier.pkl`
- `models/tfidf_vectorizer.pkl`
- `report/project_report.pdf`

Core software dependencies

- pandas >= 2.0
- scikit-learn >= 1.3
- nltk >= 3.8
- matplotlib >= 3.7
- joblib >= 1.3